МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «ЛИПЕЦКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Институт (факультет) Компьютерных наук
Кафедра Прикладной математики и системного анализа

ЛАБОРАТОРНАЯ РАБОТА №3

По дисциплине: «Современные платформы разработки ПО».
На тему: «Тестирование приложений и компонент».

Студент ПМ-24-1 (группа) _____ (подпись, дата) Сараев А.П. (фамилия, инициалы)

Руководитель

к.т.н. (ученая степень, ученое звание) _____ (подпись, дата) Немец С.Ю. (фамилия, инициалы)

Липецк 2026

Цель работы

Написание unit и интеграционных тестов с проверкой как положительных, так и отрицательных результатов.

Задание

Необходимо покрыть разработанный код unit и интеграционными тестами. Минимальный уровень покрытия – 70%. Тесты должны содержать как проверки положительных результатов, так и отрицательных. Для написания юнит-тестов можно использовать библиотеку Mockito.

Текст программы

MainTest.java

```
package org.example;

import org.example.models.User;
import org.example.services.AuthService;
import org.example.utils.FileManager;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.io.TempDir;

import java.io.ByteArrayInputStream;
import java.io.ByteArrayOutputStream;
import java.io.PrintStream;
import java.nio.file.Path;
import java.util.Map;
import java.util.Scanner;

import static org.junit.jupiter.api.Assertions.*;

public class MainTest {
    private ByteArrayOutputStream outContent;
    private PrintStream originalOut;

    @BeforeEach
    void setUp() {
        outContent = new ByteArrayOutputStream();
        originalOut = System.out;
        System.setOut(new PrintStream(outContent));
    }

    @AfterEach
    void tearDown() {
        System.setOut(originalOut);
    }

    private void provideInput(String data) {
        ByteArrayInputStream testIn = new ByteArrayInputStream(data.getBytes());
        System.setIn(testIn);
    }

    @Test
    void authenticate_ValidUser() {
        provideInput("admin\n123\n");
        Scanner sc = new Scanner(System.in);

        FileManager fileManager = new FileManager();
        Map<String, User> users = fileManager.loadUsers("users.json");
        AuthService authService = new AuthService(users);

        String username = Main.authenticate(sc, authService);

        assertEquals("admin", username);
    }

    @Test
    void authenticate_InvalidUser() {
        provideInput("ktoto\n123\n");
```

```
        Scanner sc = new Scanner(System.in);

        FileManager fileManager = new FileManager();
        Map<String, User> users = fileManager.loadUsers("users.json");
        AuthService authService = new AuthService(users);

        String username = Main.authenticate(sc, authService);

        assertNull(username);
    }

    @Test
    void main_ValidUser() {
        provideInput("admin\n123\n0\n");

        Main.main(new String[]{});

        String output = outContent.toString();

        assertTrue(output.contains("Successful authentication!"));
    }

    @Test
    void main_InvalidUser() {
        provideInput("ktoto\n123\n0\n");

        Main.main(new String[]{});

        String output = outContent.toString();

        assertTrue(output.contains("Authentication error"));
    }
}
```

MenuTest.java

```
package org.example;

import org.example.models.Laptop;
import org.example.models.LaptopType;
import org.example.models.User;
import org.example.services.AuthService;
import org.example.services.LaptopService;
import org.example.utils.FileManager;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.io.TempDir;

import java.io.ByteArrayInputStream;
import java.io.ByteArrayOutputStream;
import java.io.PrintStream;
import java.nio.file.Files;
import java.nio.file.Path;
import java.time.LocalDate;
import java.time.LocalTime;
import java.util.List;
import java.util.Map;
import java.util.Scanner;

import static org.junit.jupiter.api.Assertions.*;
```

```java
public class MenuTest {

@TempDir
Path tempDir;

private ByteArrayOutputStream outContent;
private PrintStream originalOut;

private FileManager fileManager;
private Path testLaptopsFile;

  @BeforeEach
  void setUp() throws Exception {
     outContent = new ByteArrayOutputStream();
     originalOut = System.out;
     System.setOut(new PrintStream(outContent));

     fileManager = new FileManager();

     testLaptopsFile = tempDir.resolve("testLaptops.json");
     String initialLaptops = "[]";
     Files.writeString(testLaptopsFile, initialLaptops);
  }

  @AfterEach
  void tearDown() {
     System.setOut(originalOut);
  }

  private void provideInput(String data) {
     ByteArrayInputStream testIn = new ByteArrayInputStream(data.getBytes());
     System.setIn(testIn);
  }

  private Menu createMenu_Real(String username, Scanner scanner) {
     List<Laptop> laptops = fileManager.loadLaptops("laptops.json");
     LaptopService laptopService = new LaptopService(laptops);
     return new Menu(scanner, laptopService, fileManager, username, "laptops.json");
  }

  private Menu createMenu_Temp(String username, Scanner scanner) {
     List<Laptop> laptops = fileManager.loadLaptops(testLaptopsFile.toString());
     LaptopService laptopService = new LaptopService(laptops);
     return new Menu(scanner, laptopService, fileManager, username, testLaptopsFile.toString());
  }

  @Test
  void getUserChoice_Valid() {
     provideInput("1\n");

     Scanner sc = new Scanner(System.in);
     Menu menu = createMenu_Real("admin", sc);

     assertEquals(1, menu.getUserChoice());
  }

  @Test
  void getUserChoice_Invalid() {
     provideInput("gergre\n0\n");

     Scanner sc = new Scanner(System.in);
```

```
        Menu menu = createMenu_Real("admin", sc);
        menu.getUserChoice();

        String output = outContent.toString();

        assertTrue(output.contains("Please enter a valid number"));
    }

    @Test
    void showLaptops_ZeroLaptops() {
        provideInput("1\n0\n");

        Scanner sc = new Scanner(System.in);
        Menu menu = createMenu_Real("Kira", sc);
        menu.run();

        String output = outContent.toString();

        assertTrue(output.contains("You have 0 laptops"));
    }

    @Test
    void showLaptops_NonZeroLaptops() {
        provideInput("1\n0\n");

        Scanner sc = new Scanner(System.in);
        Menu menu = createMenu_Real("admin", sc);
        menu.run();

        String output = outContent.toString();

        assertAll(
                () -> assertTrue(output.contains("Lenovo")),
                () -> assertTrue(output.contains("msi"))
        );
    }

    @Test
    void addLaptop_Successfull() {
        String input = "2\n" +
                "MacBook Pro\n" +
                "M3 Pro\n" +
                "2500\n" +
                "2024-12-25\n" +
                "14:30\n" +
                "C:\\Users\\JonyJon\\Documents\\Shared\\LSTU\\4 sem\\modern_platforms\\lab1\\anime-lucky-star-konata-
izumi-pack.png\n" +
                "GAMING\n" +
                "0\n";
        provideInput(input);

        Scanner sc = new Scanner(System.in);
        Menu menu = createMenu_Temp("admin", sc);
        menu.run();

        String output = outContent.toString();
        assertTrue(output.contains("Data saved"));

        List<Laptop> laptops = fileManager.loadLaptops(testLaptopsFile.toString());

        assertAll(
                () -> assertEquals(1, laptops.size()),
```

```
            () -> assertEquals("MacBook Pro", laptops.get(0).getModel())
        );
    }

    @Test
    void deleteLaptop_Successfull() {
        LaptopService laptopService = new LaptopService(fileManager.loadLaptops(testLaptopsFile.toString()));
        laptopService.addLaptop("admin", "Test Laptop", "Test Specs", 1000,
                LocalDate.now(), LocalTime.now(), "test.png", LaptopType.OFFICE);
        fileManager.saveLaptops(laptopService.getLaptops(), testLaptopsFile.toString());

        provideInput("3\n1\n0\n");

        Scanner sc = new Scanner(System.in);
        Menu menu = createMenu_Temp("admin", sc);
        menu.run();

        String output = outContent.toString();
        assertTrue(output.contains("Data saved"));

        List<Laptop> laptops = fileManager.loadLaptops(testLaptopsFile.toString());

        assertAll(
                () -> assertTrue(output.contains("Laptop deleted")),
                () -> assertEquals(0, laptops.size())
        );
    }

    @Test
    void editLaptop_Successfull() {
        LaptopService laptopService = new LaptopService(fileManager.loadLaptops(testLaptopsFile.toString()));
        laptopService.addLaptop("admin", "Test Laptop", "Test Specs", 1000,
                LocalDate.now(), LocalTime.now(), "test.png", LaptopType.OFFICE);
        fileManager.saveLaptops(laptopService.getLaptops(), testLaptopsFile.toString());

        String input = "4\n1\n" +
                "New Model\n" +
                "New Specs\n" +
                "1500\n" +
                "2025-01-15\n" +
                "10:00\n" +
                "new.png\n" +
                "GAMING\n" +
                "0\n";
        provideInput(input);

        Scanner sc = new Scanner(System.in);
        Menu menu = createMenu_Temp("admin", sc);
        menu.run();

        String output = outContent.toString();
        assertTrue(output.contains("Data saved"));

        List<Laptop> laptops = fileManager.loadLaptops(testLaptopsFile.toString());

        assertAll(
                () -> assertTrue(output.contains("Laptop updated")),
                () -> assertEquals(1, laptops.size()),
                () -> assertEquals("New Model", laptops.get(0).getModel())
        );
    }
```

```
    @Test
    void findLaptopByModel_Successfull() {
        provideInput("5\nLenovo\n0\n");

        Scanner sc = new Scanner(System.in);
        Menu menu = createMenu_Real("admin", sc);
        menu.run();

        String output = outContent.toString();

        assertAll(
                () -> assertTrue(output.contains("Lenovo")),
                () -> assertTrue(output.contains("i7 13000"))
        );
    }
}
```

FileManagerTest.java

```
package org.example.utils;

import org.example.models.Laptop;
import org.example.models.User;
import org.junit.jupiter.api.*;
import org.junit.jupiter.api.io.TempDir;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;
import org.mockito.Mockito;

import java.nio.file.Files;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

import static org.junit.jupiter.api.Assertions.*;

class FileManagerTest {

 private FileManager fileManager;
 Path testUsersFile;
 Path testLaptopsFile;

 @TempDir
 Path tempDir;

 @BeforeEach
 void setUp() throws Exception{
 fileManager = new FileManager();

 testUsersFile = tempDir.resolve("testUsers.json");
 String usersJson = """
 [
 {
 "username": "admin",
 "password": "123"
 },
 {
 "username": "JonyJon",
 "password": "qwerty123"
```

```
}
]
""";
Files.writeString(testUsersFile, usersJson);

testLaptopsFile = tempDir.resolve("testLaptops.json");
String laptopsJson = """
[
{
"owner" : "admin",
"model" : "Lenovo",
"specs" : "i7 13000",
                    "price" : 9999.0,
                    "date" : [ 2020, 11, 3 ],
                    "time" : [ 12, 45 ],
                    "image" : "image1",
                    "type" : "OFFICE"
                  },
                  {
                    "owner" : "JonyJon",
                    "model" : "Honor",
                    "specs" : "am5 7000",
                    "price" : 1234.0,
                    "date" : [ 2010, 3, 25 ],
                    "time" : [ 11, 31 ],
                    "image" : "image2",
                    "type" : "GAMING"
                  }
            ]
            """;
    Files.writeString(testLaptopsFile, laptopsJson);
  }

  @Test
  void encodeImage_ValidInput() {
    Assertions.assertNotEquals("",
            fileManager.encodeImage("anime-lucky-star-konata-izumi-pack.png"));
  }

  @Test
  void encodeImage_InvalidInput() {
    Assertions.assertEquals("",
            fileManager.encodeImage("image999.png"));
  }

  @Test
  void loadUsers(){
    Map<String, User> users = fileManager.loadUsers(testUsersFile.toString());

    assertAll(
            () -> assertEquals(2, users.size()),
            () -> assertEquals("admin", users.get("admin").getUsername()),
            () -> assertEquals("qwerty123", users.get("JonyJon").getPassword())
    );

  }

  @Test
  void loadLaptops_FileExistsAndNotEmpty() {
    List<Laptop> laptops;
    laptops = fileManager.loadLaptops(testLaptopsFile.toString());
```

```
        assertAll(
                () -> assertEquals(2, laptops.size()),
                () -> assertEquals("Lenovo", laptops.get(0).getModel()),
                () -> assertEquals("JonyJon", laptops.get(1).getOwner())
        );

    }

    @Test
    void loadLaptops_FileDoesntExists() {
        List<Laptop> laptops;
        laptops = fileManager.loadLaptops("notExistedLaptops.json");

        Assertions.assertEquals(0, laptops.size());
    }

    @Test
    void loadLaptops_FileExistsAndIsEmpty() throws Exception {
        Path testLaptopsEmptyFile = tempDir.resolve("testEmptyLaptops.json");
        String laptopsJson = "";
        Files.writeString(testLaptopsEmptyFile, laptopsJson);

        List<Laptop> laptops;
        laptops = fileManager.loadLaptops(testLaptopsEmptyFile.toString());

        Assertions.assertEquals(0, laptops.size());
    }

    @Test
    void saveLaptops() {
        List<Laptop> laptops = fileManager.loadLaptops("testLaptops.json");
        Path testSaveLaptopsFile = tempDir.resolve("testSaveLaptops.json");

        fileManager.saveLaptops(laptops, testSaveLaptopsFile.toString());

        List<Laptop> loadedLaptops = fileManager.loadLaptops(testSaveLaptopsFile.toString());
        Assertions.assertEquals(laptops, loadedLaptops);
    }
}
```

AuthServiceTest.java

```
package org.example.services;

import org.example.utils.FileManager;
import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.io.TempDir;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;

import java.nio.file.Files;
import java.nio.file.Path;

class AuthServiceTest {

 private AuthService authService;

 @TempDir
 Path tempDir;
```

```
@BeforeEach
void setUp() throws Exception{
Path testUsersFile = tempDir.resolve("testUsers.json");
String usersJson = """
[
{
"username": "admin",
"password": "123"
},
{
"username": "JonyJon",
                "password": "qwerty123"
              }
            ]
            """;
        Files.writeString(testUsersFile, usersJson);

        authService = new AuthService(new FileManager().loadUsers(testUsersFile.toString()));
    }

    @ParameterizedTest
    @CsvSource({
            "admin,123",
            "JonyJon,qwerty123"
    })
    void login_Successful(String username, String password) {
        boolean actual = authService.login(username, password);

        Assertions.assertTrue(actual);
    }

    @ParameterizedTest
    @CsvSource({
            "admin,1111",
            "ktoto,999",
            ","
    })
    void login_Failed(String username, String password) {
        boolean actual = authService.login(username, password);

        Assertions.assertFalse(actual);
    }
}
```

LaptopServiceTest.java

```
package org.example.services;

import org.example.models.Laptop;
import org.example.models.LaptopType;
import org.example.utils.FileManager;
import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.io.TempDir;

import java.nio.file.Files;
import java.nio.file.Path;
import java.time.LocalDate;
import java.time.LocalTime;
```

```
import java.util.List;
import java.util.NoSuchElementException;

import static org.junit.jupiter.api.Assertions.*;

class LaptopServiceTest {

    private LaptopService laptopService;

    @TempDir
    Path tempDir;

    @BeforeEach
    void setUp() throws Exception{
        Path testLaptopsFile = tempDir.resolve("testLaptops.json");
        String laptopsJson = """
                [
                    {
                      "owner" : "admin",
                      "model" : "Lenovo",
                      "specs" : "i7 13000",
                      "price" : 9999.0,
                      "date" : [ 2020, 11, 3 ],
                      "time" : [ 12, 45 ],
                      "image" : "image1",
                      "type" : "OFFICE"
                    },
                    {
                      "owner" : "JonyJon",
                      "model" : "Honor",
                      "specs" : "am5 7000",
                      "price" : 1234.0,
                      "date" : [ 2010, 3, 25 ],
                      "time" : [ 11, 31 ],
                      "image" : "image2",
                      "type" : "GAMING"
                    }
                ]
                """;
        Files.writeString(testLaptopsFile, laptopsJson);
        laptopService = new LaptopService(new FileManager().loadLaptops(testLaptopsFile.toString()));
    }

    @Test
    void getUserLaptops_RealUser() {
        String realUser = "admin";
        List<Laptop> realUserLaptops = laptopService.getUserLaptops(realUser);

        Assertions.assertAll(
                () -> Assertions.assertEquals("Lenovo", realUserLaptops.get(0).getModel()),
                () -> Assertions.assertEquals(1, realUserLaptops.size())
        );
    }

    @Test
    void getUserLaptops_FakeUser() {
        String fakeUser = "Nikolay";
        List<Laptop> fakeUserLaptops = laptopService.getUserLaptops(fakeUser);

        Assertions.assertEquals(0, fakeUserLaptops.size());
    }
```

```
    @Test
    void getLaptopByModel() {
        String realUser = "admin";

        String realLaptopModel = "Lenovo",
                fakeLaptopModel = "MSI";

        Laptop realLaptop = laptopService.getLaptopByModel(realUser, realLaptopModel),
                fakeLaptop = laptopService.getLaptopByModel(realUser, fakeLaptopModel);

        assertAll(
                () -> assertEquals(realLaptopModel, realLaptop.getModel()),
                () -> assertEquals(realUser, realLaptop.getOwner()),
                () -> assertEquals(null, fakeLaptop)
        );
    }

    @Test
    void addLaptop_Successful() {
        laptopService.addLaptop("admin", "MSI", "Baikal",
                1500, LocalDate.now(), LocalTime.now(),
                "C:\\Users\\JonyJon\\Documents\\Shared\\LSTU\\4 sem\\modern_platforms\\lab1\\anime-lucky-star-konata-
izumi-pack.png",
                LaptopType.GAMING);

        List<Laptop> laptops = laptopService.getLaptops();

        assertAll(
            () -> assertEquals(3, laptops.size()),
            () -> assertEquals("MSI", laptops.get(2).getModel())
        );
    }

    @Test
    void addLaptop_EmptyModel() {
        assertFalse(laptopService.addLaptop("admin", "", "Baikal",
                1500, LocalDate.now(), LocalTime.now(),
                "C:\\Users\\JonyJon\\Documents\\Shared\\LSTU\\4 sem\\modern_platforms\\lab1\\anime-lucky-star-konata-
izumi-pack.png",
                LaptopType.GAMING));
    }

    @Test
    void addLaptop_EmptySpecs() {
        assertFalse(laptopService.addLaptop("admin", "MSI", "",
                1500, LocalDate.now(), LocalTime.now(),
                "C:\\Users\\JonyJon\\Documents\\Shared\\LSTU\\4 sem\\modern_platforms\\lab1\\anime-lucky-star-konata-
izumi-pack.png",
                LaptopType.GAMING));
    }

    @Test
    void addLaptop_PriceNegative() {
        assertFalse(laptopService.addLaptop("admin", "MSI", "Baikal",
                -239, LocalDate.now(), LocalTime.now(),
                "C:\\Users\\JonyJon\\Documents\\Shared\\LSTU\\4 sem\\modern_platforms\\lab1\\anime-lucky-star-konata-
izumi-pack.png",
                LaptopType.GAMING));
    }

    @Test
    void deleteLaptop_Successful() {
```

```
        laptopService.deleteLaptop("admin", 0);
        List<Laptop> laptops = laptopService.getLaptops();

        Assertions.assertEquals(1, laptops.size());
        Assertions.assertEquals("JonyJon", laptops.get(0).getOwner());
    }

    @Test
    void deleteLaptop_WrongIndex() {
        assertAll(
                () -> assertFalse(laptopService.deleteLaptop("admin", 99)),
                () -> assertFalse(laptopService.deleteLaptop("admin", -10))
        );
    }

    @Test
    void deleteLaptop_WrongUsername() {
        Assertions.assertFalse(laptopService.deleteLaptop("Nikolay", 0));
    }

    @Test
    void editLaptop_Successful() {
        laptopService.editLaptop("admin", 0, "MSI", "Baikal",
                1500, LocalDate.now(), LocalTime.now(),
                "C:\\Users\\JonyJon\\Documents\\Shared\\LSTU\\4 sem\\modern_platforms\\lab1\\anime-lucky-star-konata-
izumi-pack.png",
                LaptopType.GAMING);

        List<Laptop> laptops = laptopService.getLaptops();

        assertAll(
                () -> Assertions.assertEquals(2, laptops.size()),
                () -> Assertions.assertEquals("MSI", laptops.get(0).getModel()),
                () -> Assertions.assertEquals(1500, laptops.get(0).getPrice())
        );
    }

    @Test
    void editLaptop_WrongIndex() {
        assertAll(
                () -> assertFalse(laptopService.editLaptop("admin", -1, "MSI", "Baikal",
                        1500, LocalDate.now(), LocalTime.now(),
                        "C:\\Users\\JonyJon\\Documents\\Shared\\LSTU\\4 sem\\modern_platforms\\lab1\\anime-lucky-star-konata-
izumi-pack.png",
                        LaptopType.GAMING)),
                () -> assertFalse(laptopService.editLaptop("admin", 999, "MSI", "Baikal",
                        1500, LocalDate.now(), LocalTime.now(),
                        "C:\\Users\\JonyJon\\Documents\\Shared\\LSTU\\4 sem\\modern_platforms\\lab1\\anime-lucky-star-konata-
izumi-pack.png",
                        LaptopType.GAMING))
        );
    }

    @Test
    void editLaptop_ModelEmpty() {
        assertFalse(laptopService.editLaptop("admin", 0, "", "Baikal",
                1500, LocalDate.now(), LocalTime.now(),
                "C:\\Users\\JonyJon\\Documents\\Shared\\LSTU\\4 sem\\modern_platforms\\lab1\\anime-lucky-star-konata-
izumi-pack.png",
                LaptopType.GAMING));
    }
```

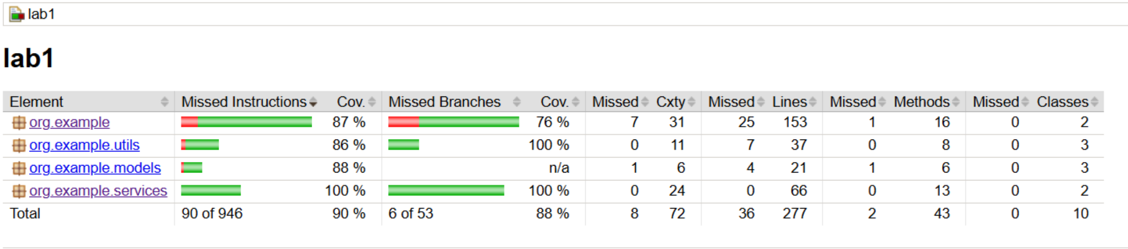
```
    @Test
    void editLaptop_SpecsEmpty() {
        assertFalse(laptopService.editLaptop("admin", 0, "MSI", "",
                1500, LocalDate.now(), LocalTime.now(),
                "C:\\Users\\JonyJon\\Documents\\Shared\\LSTU\\4 sem\\modern_platforms\\lab1\\anime-lucky-star-konata-izumi-pack.png",
                LaptopType.GAMING));
    }

    @Test
    void editLaptop_PriceNegative() {
        assertFalse(laptopService.editLaptop("admin", 0, "MSI", "Baikal",
                -1500, LocalDate.now(), LocalTime.now(),
                "C:\\Users\\JonyJon\\Documents\\Shared\\LSTU\\4 sem\\modern_platforms\\lab1\\anime-lucky-star-konata-izumi-pack.png",
                LaptopType.GAMING));
    }

    @Test
    void getLaptops() {
        List<Laptop> laptops = laptopService.getLaptops();

        Assertions.assertEquals(2, laptops.size());
        Assertions.assertEquals("Lenovo", laptops.get(0).getModel());
        Assertions.assertEquals("JonyJon", laptops.get(1).getOwner());
    }
}
```

Результаты программы на тестовых данных



lab1

lab1

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
org.example		87 %		76 %	7	31	25	153	1	16	0	2
org.example.utils		86 %		100 %	0	11	7	37	0	8	0	3
org.example.models		88 %		n/a	1	6	4	21	1	6	0	3
org.example.services		100 %		100 %	0	24	0	66	0	13	0	2
Total	90 of 946	90 %	6 of 53	88 %	8	72	36	277	2	43	0	10

Рисунок 1 — отчет Jacoco

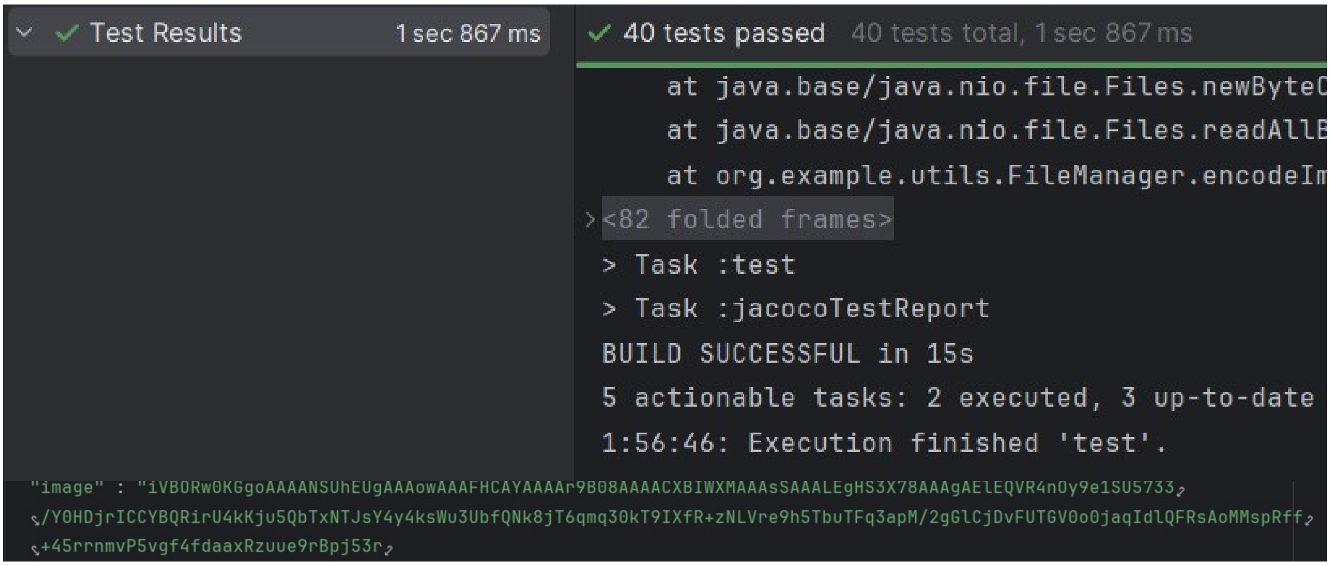


Рисунок 2 — добавление нового объекта

Вывод

В результате работы были написаны unit и интеграционные тесты с 70% покрытия кода с использованием JaCoCo для измерения покрытия кода тестами.